Introduction to Neural Networks

Multilayer Neural Networks

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

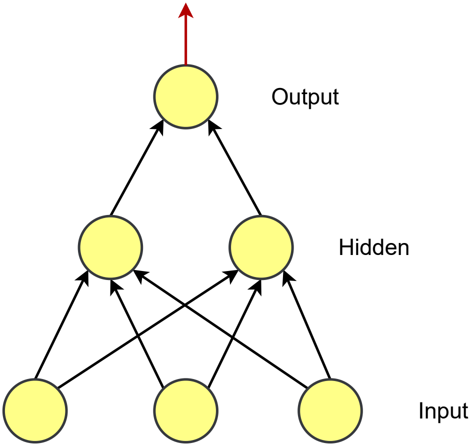
February 26, 2026©

Contents

Roadmap

1. Multilayer neural networks
2. Inference
3. Backpropagation

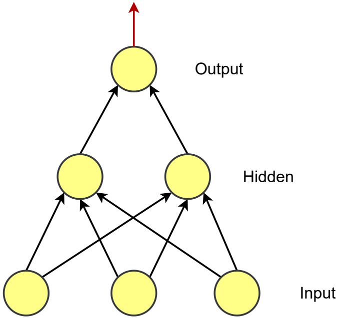
Multilayer Neural Network



Output

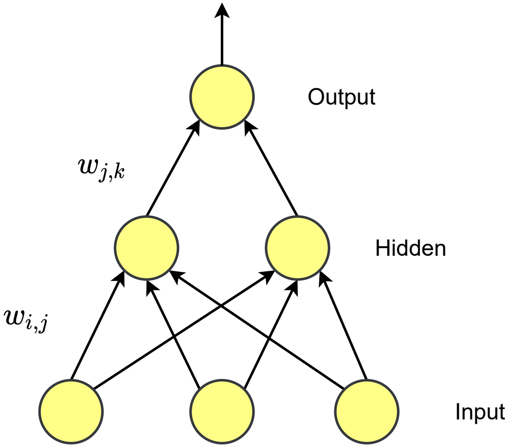
Output layer (k):

$$o_k = f\left(\sum_j w_{jk} a_j + b_k\right) \tag{1}$$



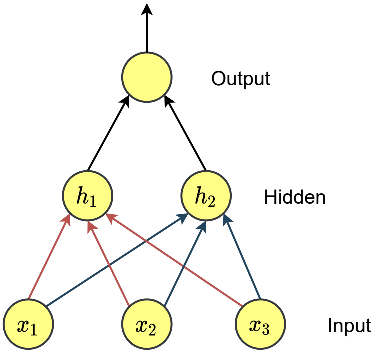
Weights

- Now the weights are labeled $w_{i,j}$, where i denotes input unit and j denotes the hidden node



Weights (2)

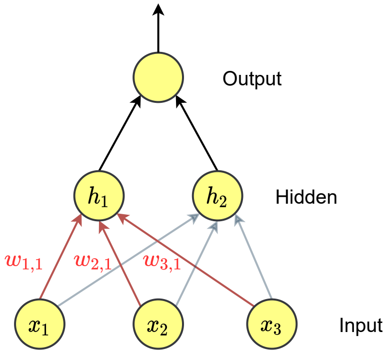
- The weights are stored in matrices



$$\begin{array}{cc} & \text{Hidden} \longrightarrow \\ \text{Input} \downarrow & \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} \end{array}$$

Weights (3)

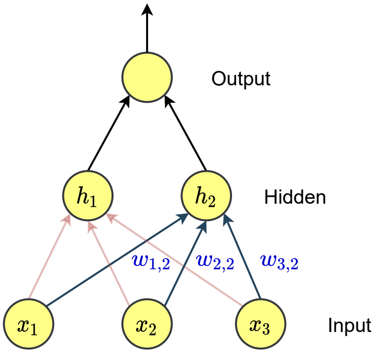
- The weights are stored in matrices



$$
\begin{array}{cc}
 & \text{Hidden} \rightarrow \\
\text{Input} \downarrow & \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix}
\end{array}
$$

Weights (4)

- The weights are stored in matrices



$$\begin{array}{cc} & \text{Hidden} \longrightarrow \\ \text{Input} \downarrow & \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} \end{array}$$

Hidden Layers

Each i-th layer is composed of:

- The weights,

$$W_i = \begin{bmatrix} w_{1,1} & \dots & w_{1,n} \\ \vdots & \ddots & \vdots \\ w_{m,1} & \dots & w_{m,n} \end{bmatrix}$$

, where m is the amount of inputs and n the outputs

- The bias,

$$B_i = [b_1, \dots, b_n]$$

- The activation function can be different for each layer, $f_i(H)$

Roadmap

1. Multilayer neural networks
2. Inference
3. Backpropagation

Inference

- From now on, we will use matrices

$$H_i = A_{i-1} W_i + B_i \tag{2}$$

e.g.

$$H_1 = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix}$$

- H_i is introduced to the activation function, then

$$A_i = f_i(H_i) \tag{3}$$

- if i is the last layer, then

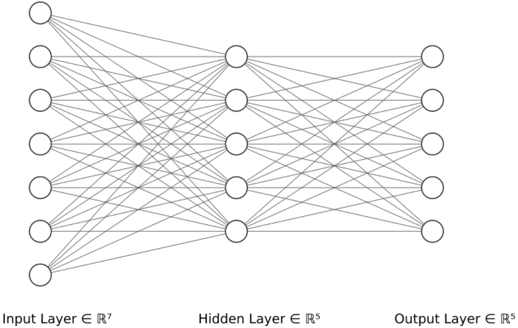
$$\hat{Y} = H_i \tag{4}$$

$$\hat{Y} = f([h_1, h_2])$$

Parameters

- Inference

$$X \to f(XW_1 + B_1) \to A_1 \to f_2(A_1 W_2 + B_2) \to \hat{Y} \tag{5}$$



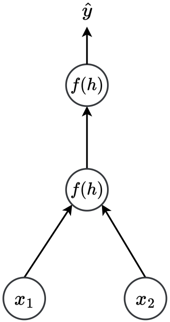
- Number of parameters in each layer:

$$c(m, n) = (m + 1)n \tag{6}$$

where m is the number of inputs and n the number of outputs.

Exercise

Tomando en cuenta la siguiente red neuronal multicapa:



- Input:

$$X = [0.1, 0.2]$$

- Layer 1:

$$W_1 = \begin{bmatrix} -0.5 \\ 0.4 \end{bmatrix}, B_1 = [0], f_1(H) = sigmoid()$$

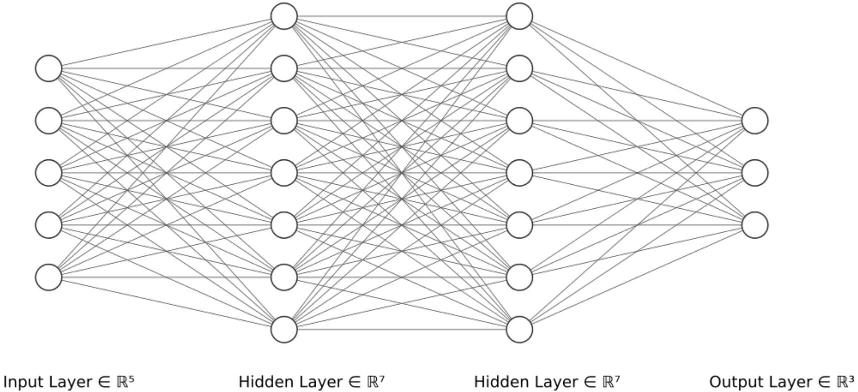
- Layer 2:

$$W_2 = \begin{bmatrix} 0.3 \end{bmatrix}, B_2 = [1], f_2(H) = tanh()$$

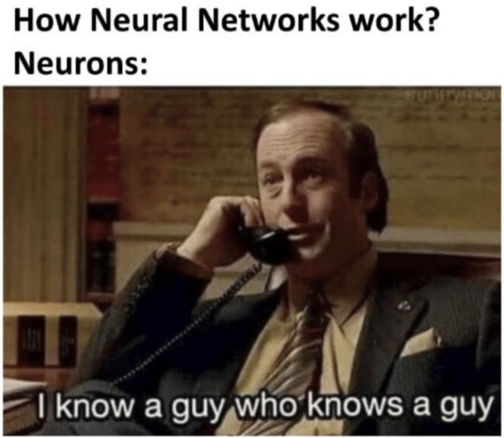
1. Calcule la salida de la capa oculta (Utilice notación matricial).
2. Calcule la predicción de la red.

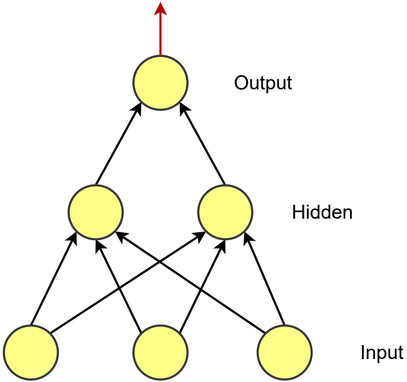
Excercises

- Determine the amount of weights required in the following network:



Multilayer network predictions

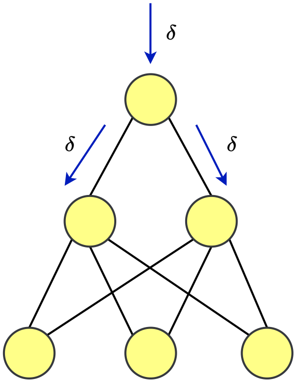




Roadmap

Backpropagation[1]

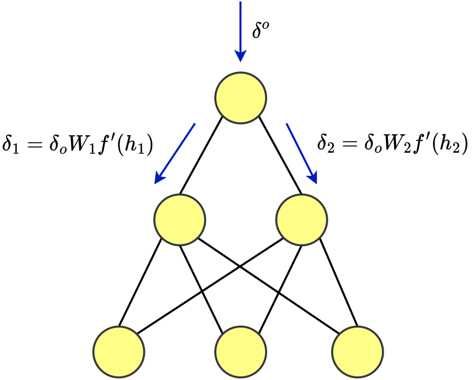
- How do we find the errors for the gradient descent step?



[1]Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088), 533-536.

Backpropagation introduction

- The errors are proportional to the error in the output layer times the weight between the units.



Ready, set...

Backpropagation rules

- It is an extension of the gradient descent
- General contribution of the weight:

$$\frac{\partial E}{\partial w_{h,o}} = \frac{\partial E}{\partial h_o}\frac{\partial h_o}{\partial w_{h,o}} = \frac{\partial E}{\partial h_o} y_h \tag{7}$$

  where o indicates the output layer and h indicates the hidden layer.

- General update rule:

$$w^{new} = w^{old} + (-1)\eta \frac{\partial E}{w^{old}} \tag{8}$$

Backprogation implementation

- In the output layer you have errors in each unit o,
- We can generalize previous error term as δ_o,
- Term error for hidden layer node h,

$$\delta_h = \sum_o w_{h,o} \delta_o f'(h_h) \tag{9}$$

- The gradient descent step is the same,

$$\Delta w_{i,h} = \eta \delta_h x_h$$

Initialization

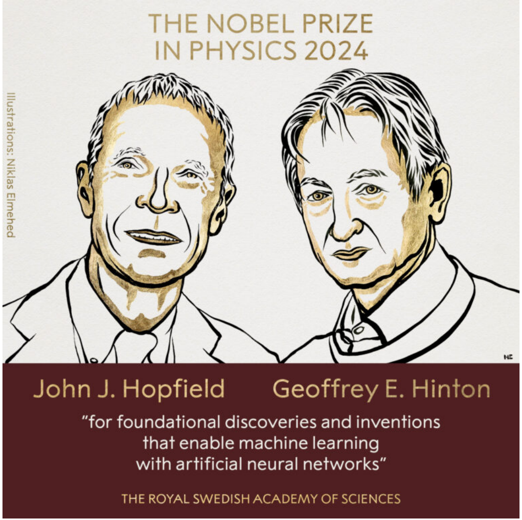
- Set the weight steps for each layer to zero
  - The input to hidden weights $\Delta w_{h,o} = 0$
  - The hidden to output eights $\Delta W_{i,h} = 0$

Backpropagation algorithm (special case)

While we are doing gradient descent for a given number of epochs.

1. For each $x \in Data$
    1. Make a forward pass, $\hat{y} = f(xW)$
    2. Calculate the error term in the output unit $\delta_o = (y - \hat{y})f'(h)$
    3. Propagate the errors to hidden layer $\delta_h = \delta_o W_h f'(h_h)$
    4. Update the weight steps:
    $\Delta w_{h,o} = \Delta w_{h,o} + \delta_o \hat{y}_h$
    $\Delta w_{i,h} = \Delta w_{i,h} + \delta_h x_i$
2. Update the weights, where η is the learning rate and m is the number of records:
$w_{h,o} = w_{h,o} + \frac{\eta}{m} \Delta w_{h,o}$
$w_{i,h} = w_{i,h} + \frac{\eta}{m} \Delta w_{i,h}$

Award



THE NOBEL PRIZE
IN PHYSICS 2024
Illustrations: Niklas Elmehed
John J. Hopfield
Geoffrey E. Hinton
"for foundational discoveries and inventions
that enable machine learning
with artificial neural networks"
THE ROYAL SWEDISH ACADEMY OF SCIENCES

Backpropagation remarks

- Backpropagation is fundamental in neural networks.
- It is already implemented in many frameworks.
- As engineers/scientists we have to understand it very well.

Backpropagation remarks

- Backpropagation is fundamental in neural networks.
- It is already implemented in many frameworks.
- As engineers/scientists we have to understand it very well.

Further readings

- IEEE Spectrum (2024) 'Nobel Prize in Physics for Machine Learning Pioneers', IEEE Spectrum, 8 de octubre. Disponible en: `https://spectrum.ieee.org/nobel-prize-in-physics` (Accedido: 4 de febrero de 2026).

References

- **Section 6.5, Back propagation and other diferentiation algorithms.** Goodfellow, Ian, et al. Deep learning. Vol. 1. No. 2. Cambridge: MIT press, 2016.
- **The back propagation algorithm.** Buduma, N., Buduma, N., & Papa, J. (2022). Fundamentals of deep learning. " O'Reilly Media, Inc.".
- **4.6 Backpropagation.** Skansi, S. (2018). Introduction to Deep Learning: from logical calculus to artificial intelligence. Springer.
- Stewart, James, Daniel K. Clegg, and Saleem Watson. Calculus: early transcendentals. Cengage Learning, 2020.

Where we are in the history?

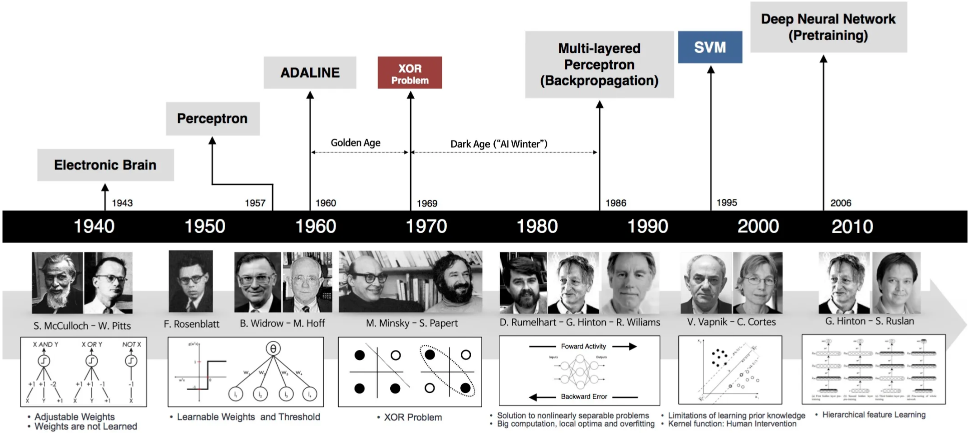


Figure: Brieft history of the neural networks [2].

[2] Sefik Ilkin Serengil, Evolution of Neural Networks, sefiks.com, Consulted: october 2024